SECURITY & READINESS ASSESSMENT

thebook Security Audit

A full-stack review of the thebookdex non-custodial spot CLOB and perpetuals program on Vara mainnet — contracts, economics, frontend, API, infrastructure, agent surface, and release governance.

DATE	COMMIT	PROGRAM	NETWORK	FINDINGS	EXPLOITS PROVEN
27 August 2026	22970f4	0x7c5dbc…2484	Vara Mainnet	41	3

Take the program out of service. Do not accept deposits.

The live mainnet program contains an **unauthenticated function that lets any account drain every token the contract holds**. It requires no admin action, no prior deposit, and no special conditions. I reproduced it against the compiled contract: an account starting at zero balance walked away with 50,000 tokens belonging to the house reserve.

Two further critical defects were also reproduced — a legacy free-money path that converts minted virtual balances into real tokens, and a perpetuals entry point that pockets a user's margin when it rejects their order.

Separately, the project's own `MAINNET.md` lists five launch-blockers, including "professional audit before launch" and "remove the Join grant". None are satisfied, yet the frontend defaults to mainnet with real bridged USDT, USDC, wETH, and wVARA.

4	9	15	13
CRITICAL	HIGH	MEDIUM	LOW

The v1 spot design is genuinely sound. Claimable-balance settlement, escrow-before-match ordering, price-time priority, and floor-division rounding that always favours the contract are all correct, and the arithmetic holds under `overflow-checks`. The failures are not in the new matching engine — they are in what was left running beside it, in what happens after an `await`, and in shipping before the checklist was finished.

SECTION 01

Findings index

ID	SEVERITY	FINDING	DOMAIN
C-01	CRITICAL	Arbitrary cross-program call drains the entire vault	Contracts
C-02	CRITICAL	Legacy virtual balances convert to real tokens on withdraw	Contracts
C-03	CRITICAL	Rejected perp open keeps the user's margin	Contracts
C-04	CRITICAL	Live on mainnet against an unfinished launch-blocker list	Governance
H-01	HIGH	Voucher endpoint is an open faucet on the sponsor account	API
H-02	HIGH	Order vector never shrinks — the book bricks at 10,000 orders	Contracts
H-03	HIGH	Market orders have no slippage bound	Contracts
H-04	HIGH	Single-key oracle with no sanity bound; stall freezes all exits	Contracts
H-05	HIGH	Admin can drain the reserve backing open positions	Contracts
H-06	HIGH	Price cache accepts unauthenticated writes	API
H-07	HIGH	Unlimited approval granted under a "this order only" label	Frontend
H-08	HIGH	No emergency pause anywhere in the system	Contracts
H-09	HIGH	Admin authority is one hot key; no multisig, no timelock	Infra
M-01	MEDIUM	Typing letters in the price field blanks the app	Frontend
M-02	MEDIUM	Spot service emits no events — no audit trail, no indexing	Contracts
M-03	MEDIUM	Open-interest cap defaults to unlimited	Economics
M-04	MEDIUM	Winning traders race each other for a shared reserve	Economics
M-05	MEDIUM	Notional uses saturating multiply instead of trapping	Contracts
M-06	MEDIUM	Rounding dust accretes permanently with no sweep	Contracts
M-07	MEDIUM	No CSP or framing protection on a wallet-signing dApp	Frontend
M-08	MEDIUM	State caps are checked before the await that can bypass them	Contracts
M-09	MEDIUM	Two high-severity advisories in shipped dependencies	Quality
M-10	MEDIUM	CI is untracked, so it never runs	Quality
M-11	MEDIUM	Agent tools have no spend limit or confirmation step	Agents

M-12	MEDIUM	Product still tells users their balances are play money	Disclosure
M-13	MEDIUM	No terms, risk disclosure, or jurisdiction statement	Disclosure
M-14	MEDIUM	Listing controls are one-way and self-attested	Contracts
M-15	MEDIUM	Tests assert error codes but never balance invariants	Quality
L-01 – L-13	LOW	Thirteen further defects — see each domain section	Various

SECTION 02

Scope and method

Everything in the repository at commit `22970f4` was reviewed: the Rust/Sails program (`app/`, ~2,300 lines across five service modules), the React frontend (~10,900 lines), the three Vercel serverless functions, nine deployment and keeper scripts, the published `thebook-sdk` and `@thebookdex/mcp` packages, CI configuration, and release documentation.

Findings were produced by manual review, then the three critical contract defects were **confirmed empirically** — I wrote exploit tests against the compiled WASM under `gtest`, ran them, and recorded the output. Those transcripts appear verbatim beside each finding. The probe file is preserved and the working tree was restored afterwards; nothing in the repository was modified.

> **What could not be verified from here.** I have no access to live chain state. Two conclusions depend on it: whether `Orderbook/SetToken` was ever called on the live program (which determines whether C-02 is exploitable today or only latent), and how much value the program currently custodies. Query both before deciding how urgently to act.

Baseline health is good and is not a finding: `cargo clippy --all-targets` is clean, `tsc -b` and `eslint` pass with no output, and all existing tests pass — 4 Rust unit tests, 52 `gtest` integration tests, and 10 frontend tests.

SECTION 03

Smart contract audit

The program deploys five services against two independent state trees: the v1 spot CLOB and perps (`SpotState`, real token custody) alongside the legacy orderbook, AMM, and perps (`DexState`, virtual balances). They share one on-chain account, and therefore **one pool of real tokens**. That sharing is the root of the two most severe findings.

C-01 Arbitrary cross-program call drains the entire vault

CRITICAL

`app/src/orderbook.rs:835` · `Orderbook/CallAgentService`

`call_agent_service(target, payload, gas_limit)` has no access control and no restriction on either argument. It sends a caller-supplied raw payload to a caller-supplied program *as the DEX itself*. An attacker points `target` at any VFT token program and supplies a `Vft/Transfer` payload naming themselves as recipient. The token program sees the DEX as the sender and moves the DEX's tokens.

This drains everything the program holds without distinction: live spot order escrow, every user's claimable balance, and the whole perps house reserve. It needs no admin action, no token registration, no deposit, and no position. The function is exposed in the deployed IDL and is callable on mainnet right now.

In my run the DEX returned `Err(AgentCallFailed)` — the reply decoder expects `Vec<u8>` and got a `bool`. That error is cosmetic. Gear has no atomic cross-program transaction, so the token transfer had already committed in the token program's own state. **The attacker keeps the tokens and the DEX reports failure.**

CARGO TEST — AUDIT_PROBE_CALL_AGENT_SERVICE_DRAINS_VAULT

```
running 1 test
AUDIT vault holds (via reserve funding) — charlie starts at 0
AUDIT CallAgentService result=Err(AgentCallFailed)  charlie balance after = 50000
test audit_probe_call_agent_service_drains_vault ... ok
```

IMPACT — Total, immediate, permissionless loss of all custodied funds.

FIX — Delete `call_agent_service` and redeploy. There is no safe version of it: any function that lets an untrusted caller choose both target and payload is equivalent to handing out the contract's signing authority. If a call-out to agents is genuinely needed later, define typed methods with a fixed target allow-list — never a raw payload.

C-02 Legacy virtual balances convert to real tokens on withdraw

CRITICAL

`app/src/orderbook.rs:114` `Join` · `:148` `SeedHouse` · `:790` `Withdraw` · `:720` `SetToken`

`Join` grants any caller `INITIAL_USD = 1,000,000,000` internal units — the testnet free-money grant — and `SeedHouse` grants the admin ten million more. `Withdraw` then reads that same internal balance and sends **real VFT tokens** out of the vault. Nothing separates the minted balance from a deposited one.

Reproduced: an account that had never deposited anything called `Join`, then `Withdraw(Usd, 50_000)`, and received 50,000 real tokens taken from the perps reserve.

CARGO TEST — AUDIT_PROBE_LEGACY_JOIN_WITHDRAWS_REAL_TOKENS

```
running 1 test
AUDIT free virtual usd granted = 1000000000
AUDIT withdraw result = Ok(50000)  charlie 0 -> 50000
test audit_probe_legacy_join_withdraws_real_tokens ... ok
```

Current exploitability. `Withdraw` returns `BadParams` while the legacy token slot is unset, and `deploy-mainnet.mjs` never calls `SetToken` — so on a program deployed by that script this is latent rather than live. That is a single admin transaction away from total loss, and it is not a mitigation: it is an accident that has not happened yet. Confirm on-chain that `Orderbook/GetTokens` returns four zero addresses before relying on it.

IMPACT — Unlimited minting of withdrawable balance; total loss the moment a legacy token is registered.

FIX — Remove `orderbook.rs`, `amm.rs`, and `perps.rs` from the deployed program entirely, along with `DexState`. `MAINNET.md` already specifies exactly this and it was not carried out. Keeping a virtual-balance ledger in the same account as real custody has no upside that justifies the risk.

C-03 Rejected perp open keeps the user's margin

CRITICAL

`app/src/perps_spot.rs:276` `open_position` — escrow at :301, rejections at :308 and :319

`open_position` escrows the margin with `vft_transfer_from().await` and *then* runs three checks that can return `Err`: the fee-versus-margin check, the market lookup, and the open-interest cap. A Sails `Err` is an ordinary reply, not a trap — and even a trap would not help, because the token program committed the transfer in a separate message. The margin lands in the contract, no position is created, and nothing is credited to the user's claim.

The open-interest path makes this routine rather than exotic. `set_market_cap` is an ordinary operational control; the moment a market reaches its cap, **every subsequent open silently confiscates the trader's margin** instead of rejecting cleanly.

CARGO TEST — AUDIT_PROBE_OI_CAP_EATS_MARGIN

```
running 1 test
AUDIT before=100000 after_open=90000 after_rejected=80000 claim=0
// second open returned Err(OiCapExceeded) — and took 10,000 anyway
test audit_probe_oi_cap_eats_margin ... ok
```

The existing test `perps_oi_cap_and_fee_revenue` covers this exact path but asserts only `res.is_err()`. It never checks the trader's balance, so the bug passes CI. See M-15.

IMPACT — Silent, permanent loss of posted margin on a rejection users cannot predict.

FIX — Move every validation before the `await`, and make the post-await path unable to fail. Where a check genuinely cannot run early, credit the escrow to `claims` before returning the error so the user can withdraw it. Adopt this as a rule: *after a successful transfer-in, the only legal exits are success or a credit-and-return.* Audit `place_limit`, `market_buy`, `market_sell`, and `fund_reserve` against it — they are currently correct and should be kept that way by test.

H-02 Order vector never shrinks — the book bricks at 10,000 orders

HIGH

`app/src/spot.rs:25, :344, :425, :558, :615` · `cancel_order` at :441

`place_limit` rejects with `BookFull` when `st.orders.len() >= 10_000`, but that vector counts every order ever placed. `cancel_order` marks status `Cancelled` without removing the entry, filled orders stay forever, and both market-order paths push a permanent record while skipping the cap check entirely. Nothing ever removes an element.

So the exchange has a hard lifetime budget of 10,000 orders, after which limit orders are permanently rejected and only market orders work. Long before that, `crossing_indices` and `get_orderbook` — both linear scans over the full vector on every single order and every book read — push gas past the block limit. The legacy orderbook got this right and removes cancelled entries with an explicit comment about this exact failure; the rewrite dropped that.

IMPACT — Guaranteed permanent denial of service after bounded usage, with gas costs degrading throughout. A griefer can reach the cap deliberately for the price of gas.

FIX — Remove filled and cancelled orders from `orders` on completion, and count only resting orders against the cap. For real depth, replace the flat vector with a price-level index (`BTreeMap<price, VecDeque<order>>` per pair) so matching and book reads stop being O(total orders ever).

H-03 Market orders have no slippage bound

HIGH

`app/src/spot.rs:575` `market_sell` · `:502` `market_buy` · `frontend/src/views/SpotTradeView.tsx:184`

`market_sell(pair_id, qty)` takes no minimum-proceeds argument. It sweeps whatever bids exist at whatever prices exist. On a curated CLOB with thin books and no committed market makers — which `MAINNET.md` names as the expected launch condition — an attacker watching the mempool pulls their resting bids and leaves a lowball, and the seller has no protection at any layer. `market_buy` bounds total spend via `max_quote` but never bounds the *price*, so a budget can be spent on a fraction of the expected base.

The frontend adds nothing: `submit` passes the raw quantity straight through, and the reference price it sizes against falls back to the off-chain oracle feed when the book is empty — the same feed that accepts unauthenticated writes (H-06).

IMPACT — Sandwich and front-running losses on every market order, unbounded on the sell side.

FIX — Add `min_quote_out` to `market_sell` and `min_base_out` to `market_buy`, reverting cleanly if unmet. Default the UI to a visible tolerance (0.5%) and surface the worst-case fill before the user signs.

H-04 Single-key oracle with no sanity bound; a stall freezes all exits

HIGH

`app/src/perps_spot.rs:237` `set_mark` · `:146` `fresh_mark` · `:29` `MARK_MAX_AGE` · `frontend/scripts/perps-keeper.mjs`

All perp PnL and every liquidation settle against a mark price that one key writes with no

validation — no deviation cap, no bounds, no second source, no TWAP. A compromised keeper sets any price, liquidates the entire book, or opens and closes around a mark it controls. The keeper is a single Node process on a Render worker holding a plaintext seed, pulling from public HTTP endpoints with no signature or staleness check on the upstream response.

The failure mode in the other direction is worse. `close_position` calls `fresh_mark`, which rejects any mark older than `MARK_MAX_AGE = 100` blocks — roughly five minutes. If the keeper stops, **nobody can close a position and nobody can liquidate one**, so positions are frozen with their margin locked until the operator restores the feed. There is no fallback close path and no operator-independent exit.

There is also no delay between opening and closing at the same mark, so anyone who sees a mark update before it lands can open ahead of it and close immediately after.

IMPACT	Total loss of all perp collateral under keeper compromise; indefinite fund lockup under keeper failure.
FIX	Bound each update to a maximum deviation from the previous mark and reject outliers. Require two independent sources to agree. Add a keeper-independent exit — after a defined staleness window, allow close-at-entry so users are never trapped. Move the keeper key to a dedicated account with no admin authority (the code currently accepts admin as a keeper).

H-05 Admin can drain the reserve backing open positions

HIGH

`app/src/perps_spot.rs:424` `withdraw_reserve` · `app/src/spot.rs:631` `transfer_admin`

`withdraw_reserve` credits the admin any amount up to the full reserve with no check against open interest. Its own doc comment concedes the problem — "the operator is responsible for leaving enough to cover open positions" — which makes solvency a matter of operator discipline rather than a contract invariant. Since `settle` caps payouts at `margin + reserve`, draining the reserve does not fail loudly; it just silently truncates what winning traders receive.

`transfer_admin` compounds it: a single transaction, no two-step accept, no timelock. One mistyped address permanently destroys the ability to list pairs, set marks, or manage the reserve.

IMPACT	Operator can withdraw funds owed to traders; a single key loss or typo is unrecoverable.
FIX	Cap withdrawable reserve at the amount above the sum of open-position worst-case liability. Make `transfer_admin` two-step (propose then accept). Put admin behind the N-of-M multisig `MAINNET.md` specifies, with a timelock on reserve withdrawal.

H-08 No emergency pause anywhere in the system

HIGH

`app/src/spot.rs, app/src/perps_spot.rs — no pause state exists`

`delist_pair` stops new orders on one pair and cannot be undone. There is no global halt for trading, withdrawals, or perps. During an incident — including any of the criticals

above — the operator has no way to stop the bleeding, and the only lever available makes the pair permanently unlistable. `MAINNET.md` lists a multisig-controlled pause as required work; it was not built.

IMPACT — An exploit in progress runs to completion. No containment, no incident response.

FIX — Add a multisig-gated global pause that blocks order placement and position opening while always leaving `cancel_order` and `withdraw` open, so a pause never traps user funds. Add `relist_pair` so delisting is reversible.

M-02 Spot service emits no events

MEDIUM

`app/src/spot.rs — #[sails_rs::service] declared with no events`

The legacy services define `OrderbookEvent` and `AmmEvent` and emit on every fill. The v1 spot service — the one handling real money — emits nothing at all. No order placed, no trade, no cancel, no withdrawal. The frontend polls `get_orderbook` every five seconds because it has no alternative, indexers cannot reconstruct history, market makers cannot subscribe to their fills, and post-incident forensics would have to be rebuilt from raw message payloads.

IMPACT — No on-chain audit trail on the money path; polling-only integrations; forensics materially harder.

FIX — Define `SpotEvent` with `OrderPlaced`, `Trade`, `OrderCancelled`, `Withdrawn`, and emit at each settlement point. Do the same for perps open, close, and liquidate.

M-05 Notional uses saturating multiply instead of trapping

MEDIUM

`app/src/spot.rs:129 notional`

`price.saturating_mul(qty)` silently returns `u128::MAX` on overflow and then divides, yielding a plausible-looking but wrong number rather than failing. This runs against the deliberate `overflow-checks = true` decision in `Cargo.toml`, whose comment argues correctly that trapping is safer than wrapping. The same reasoning applies here.

Today the values are far from the boundary — wETH at 18 decimals against USDT at 6 saturates only past roughly 10^{11} ETH — so this is a latent hazard rather than a live bug. It becomes live the moment a token with unusual decimals is listed, and `list_pair` accepts any decimals the admin types (M-14).

IMPACT — Silent mispricing instead of a clean revert, in the single function every settlement path routes through.

FIX — Use `checked_mul` and return `BadParams`, or widen to `U256` as the doc comment already anticipates.

M-06 Rounding dust accretes permanently with no sweep

MEDIUM

`app/src/spot.rs:380–392 — price-improvement refund path`

When a taker buy crosses below its limit, the escrow is split into a payout at the match price plus a refund of the difference, each floored independently. Floor division makes the two parts sum to at most the escrow — the rounding always favours the contract, which is the correct direction and means the contract is never short. But the remainder is stranded: no accounting entry references it and no function can move it.

Per fill this is sub-unit. Across millions of fills on an 18-decimal base it is still economically trivial, but it does mean the contract's token balance permanently exceeds the sum of claims plus escrow plus reserve — which will make any future solvency-monitoring alert fire on a real discrepancy that has a benign explanation.

IMPACT	Permanently locked dust; a solvency invariant that reads as inequality rather than equality.
FIX	Track accumulated dust in a counter and let the admin sweep it, or credit the residue to the maker. Either way, document the invariant as *balance ≥ claims + escrow + reserve* so monitoring can assert it.

M-08 State caps are checked before the await that can bypass them

MEDIUM

`app/src/spot.rs:344` · `app/src/perps_spot.rs:295`

`place_limit` checks `MAX_OPEN_ORDERS` and `open_position` checks `MAX_POSITIONS` before awaiting the escrow transfer. Gear processes other messages during that await, so any number of orders can pass the check concurrently and all push after it. The caps are advisory rather than enforced.

IMPACT	State-bloat bounds can be exceeded under concurrent load.
FIX	Re-check the cap after the await, on the same borrow that performs the push. In `open_position` this must credit-and-return rather than plain-return (see C-03).

M-14 Listing controls are one-way and self-attested

MEDIUM

`app/src/spot.rs:186` `list_pair` · `:226` `delist_pair`

Three related weaknesses in listing. `base_dec` and `quote_dec` are supplied by the caller and never checked against the token's own `VftMetadata/Decimals` — the doc comment notes they are "verifiable" but nothing verifies them, and a wrong value corrupts every price on that market by a factor of ten. `delist_pair` has no inverse, so delisting is permanent. And the duplicate check compares `(base, quote)` exactly, so the same asset pair can be listed twice in opposite orientations, splitting liquidity across two books.

IMPACT	An admin typo silently misprices a whole market; delisting is unrecoverable; liquidity fragments.
FIX	Read decimals from each VFT during listing and reject on mismatch. Add `relist_pair`. Reject a pair whose reverse orientation already exists.

L-01 – L-05 **Contract-level low findings** LOW

- **L-01 · Withdraw is all-or-nothing.** `spot.rs:478` takes no amount and always moves the full claim, so a user cannot leave a working balance in place or withdraw a partial amount if a large transfer keeps failing.
- **L-02 · Self-trading is permitted.** `crossing_indices` does not exclude the caller's own resting orders, so wash trading is free apart from gas. With no fees there is no direct loss, but reported volume is not trustworthy.
- **L-03 · Market orders are stored with `price: 0`.** They are correctly excluded from the book, but `get_my_orders` returns them, so order history renders a price of zero for every market fill.
- **L-04 · `set_keeper` accepts the zero address** (`perps_spot.rs:207`), silently leaving the admin as sole mark-price authority.
- **L-05 · Unbounded reads.** `get_positions`, `get_pairs`, and `get_my_orders` return unpaginated collections that scale with total state. The legacy service caps pages at `MAX_PAGE`; the v1 service does not.

SECTION 04

Economic and market design

The spot design is economically clean: a pure matching venue holding no directional risk, escrow backed 1:1 by real tokens, and no fee to argue about. The perps design is the opposite — it makes the house the counterparty to every trade with no mechanism to balance the book.

M-03 **Open-interest cap defaults to unlimited** MEDIUM

`app/src/perps_spot.rs:216 — max_oi: u128::MAX on every new market`

Every market created by `add_market` starts with an unlimited per-side cap, and `deploy-mainnet.mjs` creates the ETH and VARA markets without ever calling `set_market_cap`. The reserve's directional exposure is therefore unbounded by default and only becomes bounded if an operator remembers a separate manual step. The safe value is the default; the risky value should require an explicit decision.

There is also no funding rate. Nothing pulls open interest back toward balance, so the house carries the full one-sided exposure of a market that is, structurally, everyone against the reserve.

IMPACT	Unbounded house loss on a directional move; reserve insolvency well before any cap engages.
FIX	Require `max_oi` as an argument to `add_market` with no unlimited default. Add a funding rate proportional to OI imbalance so the market self-balances instead of relying on the reserve absorbing everything.

M-04 Winning traders race each other for a shared reserve

MEDIUM

`app/src/perps_spot.rs:98` `settle`

`settle` clamps each payout to `margin + reserve`, which correctly stops the reserve going negative. The consequence is that when the reserve is short, profit is paid first-come-first-served: whoever closes first is made whole, and whoever closes last silently receives less than they earned. There is no socialised-loss mechanism, no notification, and no way for a trader to see the reserve is thin before entering.

One reserve backs all markets, so a loss on one market truncates payouts on another. Combined with H-05, an admin withdrawal can create this condition directly.

IMPACT	A bank run on profitable positions; users who did nothing wrong are paid less than the contract computed they earned.
FIX	Expose reserve health and a coverage ratio through the read API and in the UI. Block new opens below a coverage floor. If truncation must remain possible, socialise it pro-rata rather than by transaction ordering — and say so in the risk disclosure.

L-06 – L-08 Economic low findings

LOW

- **L-06 · No spot fee and no MM incentive.** A curated venue with zero fees, no rebate, and no committed liquidity has no funded reason for a market maker to quote. `MAINNET.md` flags recruiting MMs as a launch-blocker; nothing in the repository indicates it was done, so books will be empty at launch — which is also what makes H-03 dangerous.
- **L-07 · Liquidation reward is capped by residual equity.** At `perps_spot.rs:397` the fee is `min(equity, margin × 1%)`, so a position that gaps past maintenance to zero equity pays the liquidator nothing — exactly when liquidation matters most. Nobody will run a liquidation bot for a fee that vanishes under stress.
- **L-08 · Maintenance margin is 0.5% against 20× leverage.** A 20× position is liquidatable at roughly a 4.75% adverse move, with the 0.5% buffer covering the gap. On a five-minute-stale mark that buffer is thin, and every shortfall lands on the reserve.

SECTION 05

Backend and API audit

Three Vercel serverless functions. All three set `Access-Control-Allow-Origin: *`, none authenticate, and none rate-limit. Two of them spend money.

H-01 Voucher endpoint is an open faucet on the sponsor account

HIGH

`frontend/api/voucher.ts`

`POST /api/voucher` accepts any address and issues a gas voucher funded from a live

mainnet sponsor account. There is no authentication, no rate limit, no proof of work, no origin restriction, and no per-IP cap — the only validation is that the address decodes. Addresses are free to generate, so the cost of draining the sponsor is one HTTP request per voucher. `VOUCHER_VARA=5` is configured, five times the code's own documented default of 1.

The vouchers are scoped to the DEX program and the four token programs, which means the endpoint also **sponsors the gas for attacking the contract**. An attacker exploiting C-01 does not need to fund an account first.

IMPACT	Direct drain of the sponsor's mainnet balance; free gas for attacks against the DEX.
FIX	Require a signed challenge proving control of the address, rate-limit per address and per IP with durable storage, cap total daily issuance, and restrict CORS to the app's own origin. `MAINNET.md` decided v1 users pay their own gas — if that still holds, delete the endpoint.

H-06 Price cache accepts unauthenticated writes

HIGH

`frontend/api/prices.ts — POST branch`

The `POST` handler writes caller-supplied `prices` and `history` straight into the shared Vercel KV key `thebookdex:prices`, with no authentication and no validation beyond a truthiness check. That cache is served to every visitor whenever live feeds are unavailable. Anyone can push arbitrary prices and a fabricated 200-point history to all users.

This is not purely cosmetic. The frontend's `effPrice` falls back to this feed when the book is empty, and it drives the percentage-of-balance sizing chips and the order-cost estimate — so a poisoned price shapes what users enter into an order form that has no slippage guard (H-03).

IMPACT	Display and order-sizing manipulation for all users; fabricated price history.
FIX	Delete the `POST` branch — the `GET` path already fetches live prices and populates the cache itself. If a writer is genuinely needed, require a shared secret and validate every value against a plausibility band.

M-16 LLM endpoint is an unmetered proxy to a paid key

MEDIUM

`frontend/api/agent.ts`

`POST /api/agent` forwards to Groq using a server-held key with open CORS and no rate limit, so anyone can run inference on the project's quota from any origin. User-supplied `opportunities[].title` and `.rationale` are interpolated directly into the system prompt with no sanitisation, giving a straightforward prompt-injection path into text the UI presents as the user's agent speaking.

The system prompt is also materially wrong on a mainnet product — it instructs the model that this is "a simulated on-chain DEX on the Vara testnet (balances are play-money, no real funds)". See M-12.

IMPACT Quota theft and cost abuse; injected content attributed to the product.

FIX Restrict CORS to the app origin, rate-limit per session, cap request size, and pass user strings as clearly delimited data rather than prompt text. Correct the system prompt to describe real funds on mainnet.

L-09 – L-10 API low findings

LOW

- **L-09 · Voucher handler robustness.** The `GearApi` promise is cached at module scope and never re-created, so a dropped connection wedges the function until the instance recycles. `signAndSend` has no timeout and can hang to the platform limit. Seed detection routes a `0x`-prefixed seed to `addFromMnemonic`, which will throw. Failures return HTTP 200 with `enabled: true` and the raw error string.
- **L-10 · Serverless module state.** `memCache`, `liveCache`, and `varaCache` in `prices.ts` are per-instance, so cache behaviour varies unpredictably across concurrent invocations. Two Redis clients are declared as dependencies but neither is used — the handler calls the KV REST API directly.

SECTION 06

Frontend audit

The frontend is in good shape structurally — clean TypeScript, no lint errors, sensible hook decomposition, correct use of `withAddress` so per-caller queries resolve to the connected account. The problems are concentrated in input handling and consent.

H-07 Unlimited approval granted under a "this order only" label

HIGH

`frontend/src/views/SpotTradeView.tsx:400` · `frontend/src/components/ui/AllowanceGate.tsx`

On a buy, the approve button sends `MAX_U128` — an unlimited allowance on the quote token. The label directly beneath it reads: *"One-time approval lets the exchange escrow your {symbol} for this order."* The user is told the approval is scoped to one order and grants an unlimited one. The sell path correctly approves only the amount required, which makes the buy path look like an oversight rather than a decision.

An unlimited standing allowance is only as safe as the contract holding it, and this contract has C-01 — so the exposure is not the order size, it is the user's entire USDT and USDC balance.

IMPACT Users grant unlimited spending authority without informed consent, to a contract with a live drain path.

FIX Approve exactly what the order needs, matching the sell path. If a standing allowance is offered as a convenience, make it an explicit opt-in with its own copy and a visible revoke control.

M-01 Typing letters in the price field blanks the app

MEDIUM

`frontend/src/lib/units.ts` `parseUnits` · `SpotTradeView.tsx:58–60` · no error boundary in the tree

`parseUnits` passes its input to `BigInt()` with no validation. `SpotTradeView` calls it in the component body, not inside a handler, and the application has no React error boundary anywhere. So a throw unmounts the entire tree to a blank page.

```
NODE — PARSEUNITS(VALUE, 6)

"1.25"  -> 1250000
"abc"   -> THROWS SyntaxError: Cannot convert abc000000 to a BigInt
"1e3"   -> THROWS SyntaxError: Cannot convert 1e3000000 to a BigInt
"1 000" -> THROWS SyntaxError: Cannot convert 1 000000000 to a BigInt
"0x10"  -> 268435456  // parsed as hex — silently wrong, no error
"1.2.3" -> 1200000  // third component dropped silently
```

The `0x10` case is the one that matters most: it does not crash, it silently returns 268.435456 for what the user typed as a small number. `inputMode="decimal"` is a keyboard hint, not a constraint — paste, desktop keyboards, and autofill all bypass it.

IMPACT — Total UI crash from ordinary mistyping; silent numeric corruption on the order-entry path.

FIX — Validate against `/^\d*\.?\d*$/` before conversion and return `0n` otherwise; reject multiple decimal points. Add an error boundary around the router so no single component can blank the app. Add unit tests for `parseUnits`, `formatUnits`, and `notional` — none of the three is currently tested.

M-07 No CSP or framing protection on a wallet-signing dApp

MEDIUM

`frontend/vercel.json — rewrites only, no headers block`

The deployment sets no security headers at all: no `Content-Security-Policy`, no `frame-ancestors`, no `X-Content-Type-Options`, no `Referrer-Policy`. Any site can iframe the app and overlay it, which for an interface whose whole purpose is prompting wallet signatures is the textbook clickjacking setup. A CSP would also be the last line of defence against injected script reaching the wallet extension.

IMPACT — Clickjacking against signature prompts; no mitigation layer if script injection occurs.

FIX — Add a `headers` block to `vercel.json` with `frame-ancestors 'none'`, a CSP restricting `connect-src` to the Vara RPC and the app's own API, `X-Content-Type-Options: nosniff`, and `Referrer-Policy: strict-origin-when-cross-origin`.

L-11 – L-13 Frontend low findings

LOW

- **L-11 · Trading inputs are unlabelled.** Price and amount use a `<span>` for their label with no `htmlFor`, no `id`, and no `aria-label`, so a screen reader announces the money form as unlabelled edit fields. Most view components contain no ARIA attributes at all.

- **L-12 · Uncommitted fix is half-applied.** In the working-tree change to `MarketDataProvider.tsx:441`, state is updated from the new `next` object but `appendHistory(merged)` on the following line still uses the stale snapshot the fix exists to avoid — so history keeps recording the null VARA the fix removes from state.
- **L-13 · Dead PWA cache rule.** The `runtimeCaching` entry matches `https://rpc.vara.network`, but the app connects over WebSocket, which service workers do not intercept. The rule never fires.

SECTION 07

Infrastructure, keys, and governance

C-04 Live on mainnet against an unfinished launch-blocker list

CRITICAL

`MAINNET.md — "Launch-blockers (must be true before mainnet)"`

The project wrote its own release gate and then shipped past it. All five blockers are unmet:

- **Audited custody contract** — no audit; this document is the first review, and it found three exploitable criticals.
- **Multisig live as admin** — admin is the deployer key, still single-signature.
- **At least one committed market maker** — no evidence of one; books will be empty.
- **All testnet endpoints and "free money" copy removed** — see M-12; the free-money code is not merely described, it is still deployed (C-02).
- **Stablecoin address confirmed and pinned** — this one is genuinely done, in `docs/mainnet-addresses.md`.

Workstream A separately requires removing the `Join` grant, removing the virtual house stockpile, an emergency pause, and a reentrancy review of the async VFT paths. Those map exactly onto C-02, H-08, and C-03. The plan identified the right risks; the work was not completed before deploying.

IMPACT	Real user funds are exposed to a contract the project's own process says should not yet be live.
FIX	Treat the blocker list as binding. Take the program out of service, complete workstream A, obtain an independent audit, then redeploy.

H-09 Admin authority is one hot key; no multisig, no timelock

HIGH

`render.yaml · frontend/.env · deploy and keeper scripts`

One seed is the DEX admin, the listing authority, the mark-price keeper, and the reserve controller. It lives as an environment variable on an always-on Render worker and in a plaintext `frontend/.env` on a development machine alongside a live Groq API key. The `.env` file is correctly gitignored and I confirmed no secret has ever been committed —

but a single compromise of either location yields full control of listings, marks, and the reserve.

`render.yaml` compounds it: the worker carrying that seed is configured with `NODE_ADDRESS: wss://testnet.vara.network` while the frontend runs on mainnet. That worker is also documented as pushing marks and quoting house liquidity for the *legacy* virtual-balance services — the ones C-02 says should not exist.

The same pattern repeats across scripts: `market-runner.mjs`, `agent-keeper.mjs`, `fund-reserve.mjs`, and `deploy-agent.mjs` all default `NODE_ADDRESS` to testnet. Only `deploy-mainnet.mjs` guards against a stray `.env` redirecting it. Running `fund-reserve.mjs` without setting the variable sends a real admin action to the wrong chain.

IMPACT — Single point of total compromise; operational scripts default to the wrong network while holding the admin key.

FIX — Stand up the N-of-M multisig and transfer admin to it. Split the keeper into its own key with no admin rights. Move secrets to a managed store. Make `NODE_ADDRESS` required with no default in every script that signs, and apply the CLI-wins guard from `deploy-mainnet.mjs` everywhere. Retire the legacy market-runner with the legacy services.

M-17 No monitoring, alerting, or incident runbook

MEDIUM

`Repository-wide — no monitoring configuration present`

`MAINNET.md` calls for alerting on abnormal withdrawals, stuck escrow, and contract errors, plus an incident runbook. Neither exists. With no events emitted (M-02) there is nothing to monitor even if a monitor were built, and with no pause switch (H-08) there would be no response available. An exploit of C-01 would be discovered when a user noticed their balance was gone.

IMPACT — No detection and no response capability for fund-loss events.

FIX — After adding events, run a solvency monitor asserting *token balance ≥ claims + escrow + reserve* per token every block, alert on any withdrawal above a threshold, and write the runbook naming who pauses and how.

SECTION 08

SDK and agent surface

Two published packages give autonomous agents direct mainnet trading authority: `thebook-sdk` and the `@thebookdex/mcp` tool server. Both are honest about running on mainnet with real funds — the MCP README says so plainly. The gap is that the safety model described in the plan was never built.

M-11 Agent tools have no spend limit or confirmation step

MEDIUM

`mcp/server.mjs · sdk/thebook.mjs:196`

`MAINNET.md` workstream D specifies "on-chain spend limits and/or a keeper-authorized / scoped-session model so a compromised agent key can't drain the owner". No such mechanism exists in the contract, so the agent's key is unrestricted authority over its wallet. The MCP server exposes `thebook_approve`, `thebook_market_buy`, and `thebook_market_sell` with no cap, no confirmation, and no dry-run, and the SDK's own guidance is to "approve a large amount to avoid re-approving every trade".

The decimal spread makes model error expensive: wETH is 18 decimals, wVARA 12, and the stablecoins 6, all passed as raw smallest-unit integers. A model that reaches for the wrong exponent submits an order a million times too large, into market functions that have no slippage bound (H-03).

IMPACT — A single hallucinated exponent or a leaked agent seed can empty the agent's wallet.

FIX — Add on-chain per-session spend limits as planned. Meanwhile, enforce a configurable per-trade and per-day cap in the MCP server, have tools echo the human-readable amount and require confirmation above a threshold, and reverse the approval guidance to per-order.

L-14 SDK gasless path silently depends on the open faucet

LOW

`sdk/thebook.mjs:97` `ensureVoucher`

`ensureVoucher` tries once, caches the result forever, and swallows every error. An agent that starts while the voucher endpoint is briefly down pays its own gas for the rest of its lifetime with no signal, and the README recommends preferring the voucher over funding the account — which points agents at the endpoint in H-01.

SECTION 09

Testing, dependencies, and CI

M-15 Tests assert error codes but never balance invariants

MEDIUM

`tests/gtest.rs` · `app/src/spot.rs tests` · `frontend/src/lib/*.test.ts`

The suite is substantial — 52 `gtest` integration tests covering the spot and perps happy paths, refunds, and delisting. But it tests that rejections *return the right error*, not that they *leave balances correct*. `perps_oi_cap_and_fee_revenue` exercises the exact code path as C-03 and asserts only `res.is_err()`; adding two balance assertions to that same test turns it red, which is how I confirmed the bug.

The gaps line up with the findings. Only 4 unit tests exist in the contract, all in `spot.rs` and none touching settlement. No test asserts the solvency invariant. No test drives the order vector toward `MAX_OPEN_ORDERS` (H-02). Nothing exercises `call_agent_service` for abuse — the one test that touches it only checks a nonexistent target. On the frontend, 10 tests cover formatting and error mapping, while `units.ts`, which converts every user input into an on-chain amount, has none.

IMPACT — Exploitable defects pass a green suite; the money paths have the thinnest coverage in the codebase.

FIX — Assert balances on every rejection path, not just error codes. Add a solvency invariant checked after each test. Add property tests over the matching engine for the conserved quantity. Add a state-growth test that runs the order vector to its cap. Test `units.ts`.

M-10 CI is untracked, so it never runs

MEDIUM

`.gitignore:12 · .github/workflows/ci.yml — present on disk, removed from git in 16b1a12`

`.github/workflows/` is gitignored and the workflow was deliberately untracked ("remove ci from tracking — push workflow scope needed"). The file exists locally and never runs on the repository, so `fmt`, `clippy -D warnings`, and `cargo test` gate nothing. Even restored, it covers only Rust: no frontend build, lint, or test; no `npm audit`; no secret scanning; and no `cargo deny`.

IMPACT — No automated gate on a codebase handling real funds.

FIX — Restore the workflow with the token scope it needs, and extend it: `tsc`, `eslint`, `vitest`, `npm audit`, secret scanning, and `cargo deny`.

M-09 Two high-severity advisories in shipped dependencies

MEDIUM

`frontend — npm audit --omit=dev`

`nanoid` 4.0.0–5.1.15 (GHSA-28wg-ghj8-5hjv, non-secure generators can loop indefinitely on a negative size) reaches production through `@gear-js/react-hooks`. The advised fix downgrades that package across a breaking change, so this needs a considered upgrade rather than `audit fix --force`. Rust dependencies are pinned tightly and clean, though `trie-db` 0.29.1 carries a future-incompatibility warning worth tracking against the `rust-toolchain.toml` pin at 1.95.0.

IMPACT — Known-vulnerable transitive code in the production bundle.

FIX — Upgrade `@gear-js/react-hooks` to a release carrying a patched `nanoid`, or pin an override. Add `npm audit` to CI so this surfaces automatically.

SECTION 10

Disclosure and compliance readiness

Not legal advice — these are gaps a reviewer would raise, to take to counsel.

M-12 Product still tells users their balances are play money

MEDIUM

`frontend/api/agent.ts:46 · README.md:7,11 · DEPLOY.md · skills.md:95`

The most serious instance is user-facing: the agent brief's system prompt instructs the model that this is "a simulated on-chain DEX on the Vara testnet (balances are play-money, no real funds)". The model is asked to write in the user's own voice, so a user reading that brief on a mainnet exchange is being told, in the product's voice, that their real money is not real.

Around it, `README.md` carries a testnet badge and states the project is "preparing for Vara testnet launch"; `DEPLOY.md` is testnet end to end; `skills.md` gives testnet as the network for the published agent skill pack. `MAINNET.md` flagged this exact copy as "dangerously wrong" once real money is involved.

IMPACT	Users and agents materially misled about whether funds are real, in the product itself.
FIX	Correct the system prompt first — it is the only instance users read. Then update README, DEPLOY, and skills.md to mainnet, and audit every remaining testnet reference.

M-13 No terms, risk disclosure, or jurisdiction statement

MEDIUM

`frontend/src — no terms, risk, or disclosure copy anywhere`

The interface offers 20× leveraged perpetual futures with no terms of service, no risk disclosure, no statement of who operates the venue, and no jurisdiction or eligibility language. Nothing tells users the reserve can truncate their payout (M-04), that positions become unclosable if the keeper stalls (H-04), or that the contract is unaudited.

`MAINNET.md` notes derivatives regulation was deferred to v2 alongside perps — but perps shipped in v1. That deferral no longer matches what is deployed, and offering leveraged derivatives is the part most likely to attract regulatory attention.

IMPACT	Regulatory and liability exposure; users cannot assess risks the design deliberately accepts.
FIX	Publish terms and a risk disclosure covering leverage, liquidation, reserve solvency, keeper dependency, and audit status. Take the deferred derivatives review to counsel now that perps are live.

SECTION 11

Remediation roadmap

Sequenced by what stops loss soonest. Phase 0 is measured in hours, not sprints.

PHASE 0 Contain

Today

- Take the frontend offline or replace it with a notice. Every hour it invites deposits into a drainable contract.
- Query `Orderbook/GetTokens` on the live program. If any slot is non-zero, C-02 is live and the

Query `orderbook/getTokens` on the live program. If any slot is non-zero, C-02 is live and the reserve should be moved out first.

- Withdraw the perps reserve to a safe account. It is the largest single balance exposed to C-01.
- Disable `/api/voucher` — remove `SPONSOR_SEED` from Vercel, which makes the endpoint inert by design.
- Rotate the Groq key and the sponsor seed; both have sat in a plaintext dotfile.
- Notify anyone holding a claim balance or an open position, with instructions to withdraw and close.

PHASE 1 Fix the criticals

This week

- Delete `call_agent_service` — C-01.
- Remove `orderbook.rs`, `amm.rs`, `perps.rs`, and `DexState` from the program — C-02. This also retires the legacy market-runner.
- Move all validation before the escrow await in `open_position`; adopt the credit-and-return rule and test every rejection path for balance conservation — C-03.
- Add the multisig-gated pause, leaving cancel and withdraw always open — H-08.
- Prune completed orders and count only resting ones against the cap — H-02.
- Add `min_quote_out` / `min_base_out` to the market functions — H-03.

PHASE 2 Harden

Before redeploy

- Emit events across every spot and perps settlement path — M-02.
- Bound mark-price deviation, require two sources, add a keeper-independent exit — H-04.
- Cap reserve withdrawal against open liability; make `transfer_admin` two-step — H-05.
- Require `max_oi` at market creation; design the funding rate — M-03.
- Verify token decimals on-chain at listing; add `relist_pair` — M-14.
- Approve per-order in the UI; validate `parseUnits`; add an error boundary — H-07, M-01.
- Delete the `prices.ts` POST branch; restrict CORS; rate-limit `agent.ts` — H-06, M-16.
- Add security headers to `vercel.json` — M-07.
- Restore and extend CI; resolve the `nanoid` advisory — M-10, M-09.

PHASE 3 Satisfy the launch gate

Before real funds return

- Stand up the N-of-M multisig and transfer admin to it; split out the keeper key — H-09.
- Independent professional audit of the reduced program. This document is a strong starting brief, not a substitute.
- Testnet dress rehearsal of the fixed contract with real VFT tokens.
- Solvency monitoring, withdrawal alerting, and a written incident runbook — M-17.
- Publish terms and risk disclosure; correct all testnet and play-money copy — M-13, M-12.
- Add agent spend limits before promoting the SDK and MCP packages further — M-11.
- Confirm at least one committed market maker, as the project's own blocker list requires.

SECTION 12

Reproduction

The three exploit tests are preserved in the session scratchpad as `audit-probes.rs`. To re-run them, append the file to `tests/gtest.rs`, add `const CHARLIE: u64 = 7;` beside the existing actor constants, then:

REPRODUCE

```
cargo test --test gtest audit_probe -- --nocapture

test audit_probe_call_agent_service_drains_vault ... ok
test audit_probe_legacy_join_withdraws_real_tokens ... ok
test audit_probe_oi_cap_eats_margin ... ok
```

Each probe asserts the exploit *succeeds*, so all three passing is the finding. Once the fixes in Phase 1 land, invert the assertions and keep them as regression tests — a green suite should then mean the attacker gets nothing.

The working tree was restored after the run; `git status` shows only the two frontend files that were already modified before this review began.